INSTITUTO TECNOLÓGICO DE MÉXICO

CAMPUS ORIZABA

“PRACTICA 2”

EQUIPO 9

GARCIA CEDEÑO IRVING EMANUEL

ROJAS BONILLA SANTIAGO

MARTINEZ ROSAS JOSE ANGEL

MATERIA: PROGRAMACION ORIENTADA A OBJETOS

GRUPO: VESPERTINO 1-2 PM

MAESTRO: RAFAEL HERRERA GARCIA

FECHA DE ENTREGA: 7 DE MARZO DE 2026

Problema planteado:

Para el manejo de las mercancías en cierto restaurant requieren un programa que permita gestionarlas, de cada mercancía, en el almacén se tiene registro de nombre, existencia, fecha de ultima entrada al almacén, punto de reorden (cantidad mínima cuando se tiene que elaborar un pedido al proveedor), cada mercancía en el almacén puede ser pedido a más de un proveedor, de cada proveedor se tiene su nombre, teléfono y dirección. Cuando de una mercancía se requiere hacer un pedido se identifica el proveedor y se registra la cantidad pedida y la fecha del pedido, de una mercancía puede tener varios pedidos en distintas fechas, cuando un pedido llega al almacén, se registra cantidad que llega, fecha de llegada y se actualizan las existencias.

Análisis del problema: el problema nos pide registrar la mercancía del almacén de un restaurante de cada mercancía nos pide registrar su nombre, existencia, fecha de ultima entrega en el almacén y punto de reorden, nos indica que a cada mercancía puede ser pedida a mas de un proveedor, ósea que podemos tener una lista de proveedores, de cada proveedor debemos registrar su nombre, teléfono, dirección, también nos indica que cuando se hace un pedido de mercancía se identifica el proveedor y se registra la cantidad pedida y la fecha del pedido, nos recalca que la mercancía puede tener varios pedidos en distintas fechas, y que cuando una mercancía llega al almacén se registra la cantidad que llego y la fecha de llegada y se actualizan las existencias de la mercancía, esto quiere decir que necesitamos una lista de pedidos donde se registren todos estos datos.

Clase Mercancía:

Mercancía			
Atributo	**Tipo de dato**	**Cardinalidad**	**Obligatoriedad**
Nombre	String	1	si
Existencia	int	1	si
FechaUltimaEtregaAlmacen	LocalDate	1	si
PuntorReorden	int	1	si

Operaciones Clase Mercancía:

Mercancia			
Operación	**Tipo de dato (retorno)**	**Argumentos**	**Alcance**
AgregarProveedor	String		public
RegistrarPedido	LocalDate		public
ActualizarExistencia	int		public
necestiaReorden	boolean		public

Clase Proveedor:

Proveedor			
Atributo	**Tipo de dato**	**Cardinalidad**	**Obligatoriedad**
Nombre	String	1	Si
Teléfono	String	1	si
Dirección	String	1	si

Operaciones clase proveedor:

Proveedor			
Operación	**Tipo de dato (retorno)**	**Argumentos**	**Alcance**
MostrarDatos	void		public

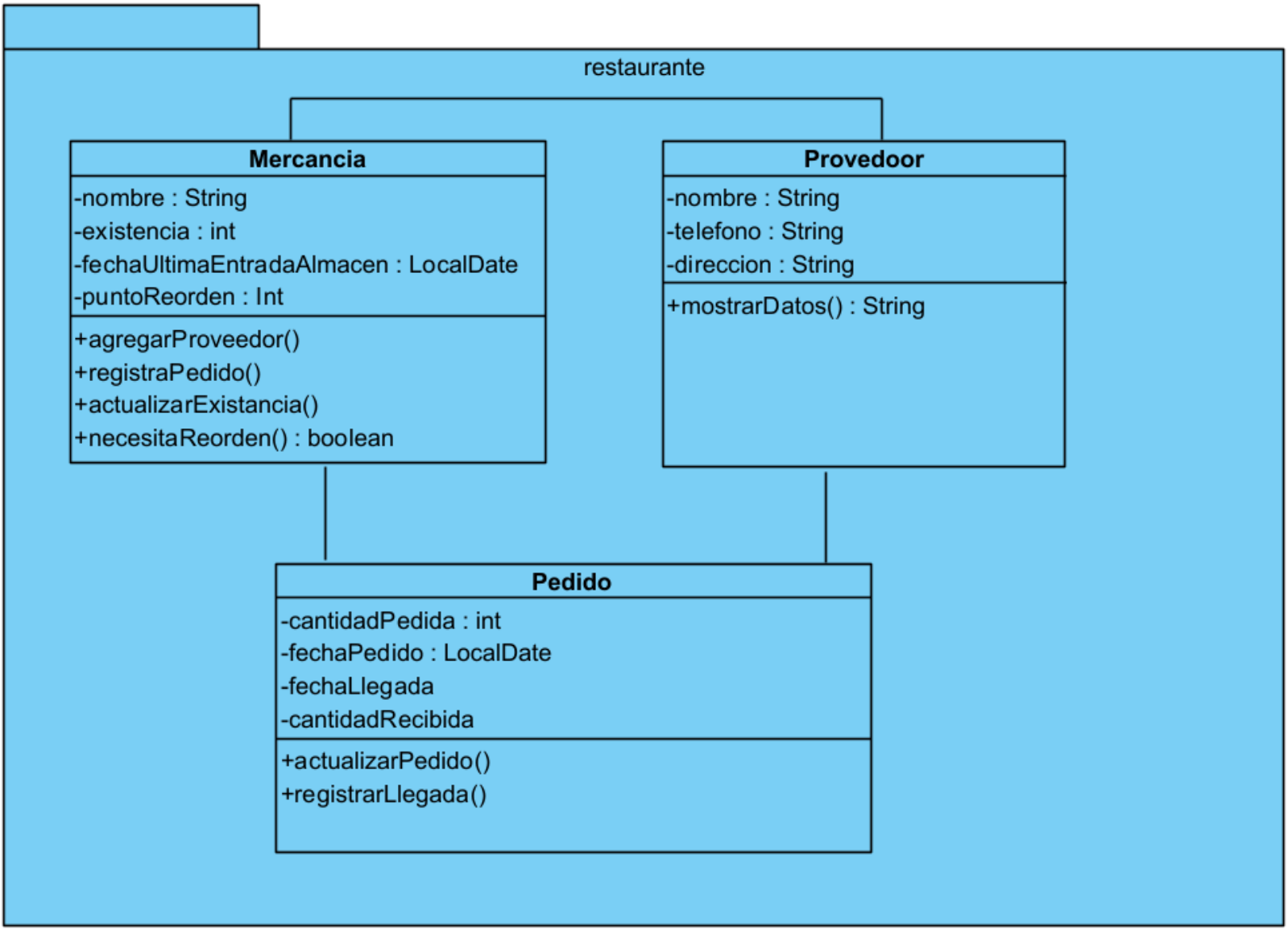
Clase Pedido:

Pedido			
Atributo	**Tipo de dato**	**Cardinalidad**	**Obligatoriedad**
CantidadPedida	int	1	Si
FechaPedido	LocalDate	1	si
FechaLlegada	LocalDate	1	si
CantidadRecibida	int	1	si

Operaciones Clase Pedido:

Pedido			
Operación	**Tipo de dato (retorno)**	**Argumentos**	**Alcance**
ActualizarPedido	void	FechaPedido, CantidadPedida	public
RegistrarLlegada	void		public

Diagrama UML:



Codigo Clase Mercancia:

mercancia.java × pedido.java proveedor.java

```java
package Almacen.Restaurante;
import java.time.LocalDate;
import java.util.ArrayList;

public class mercancia {  no usages  new *

    private String nombre;  1 usage
    private int existencia;  5 usages
    private LocalDate fechaUltimaEntregaAlmacen;  2 usages
    private int puntoDeReorden;  2 usages
    private ArrayList <proveedor> proveedores;  2 usages
    private ArrayList <pedido> pedidos;  2 usages

    public mercancia(String nombre, int existencia, LocalDate fechaUltimaEntregaAlmacen, int puntoDeReorden, ArrayList proveedores, ArrayList
        this.nombre = nombre;
        this.existencia = existencia;
        this.fechaUltimaEntregaAlmacen = fechaUltimaEntregaAlmacen;
        this.puntoDeReorden = puntoDeReorden;

        this.proveedores = new ArrayList();
        this.pedidos = new ArrayList();
    }

    public  String agregarProveedor(proveedor p) {  no usages  new *
        proveedores.add(p);
```

```java
public class mercancia {  no usages  new *

    public  String agregarProveedor(proveedor p) {  no usages  new *
        proveedores.add(p);
        return "Proveedor agregado";

    }

    public LocalDate agregarPedido (pedido p) {  no usages  new *
        pedidos.add(p);
        return p.getfechaPedido();
    }

    public int actualizarExistencias(int cantidad, LocalDate fechaLlegada){  no usages  new *

        existencia = existencia + cantidad;
        fechaUltimaEntregaAlmacen = fechaLlegada;
        return existencia;
    }

    public boolean necestiaReorden(){  no usages  new *

        return existencia <= puntoDeReorden;
    }
```

```java
public class mercancia {  no usages  new *
    public int actualizarExistencias(int cantidad, LocalDate fechaLlegada){  no usages  new *
    }

    public boolean necestiaReorden(){  no usages  new *

        return existencia <= puntoDeReorden;
    }


    @Override  new *
    public String toString() {
        return "mercancia{" +
                "nombre='" + nombre + '\'' +
                ", existencia=" + existencia +
                ", fechaUltimaEntregaAlmacen=" + fechaUltimaEntregaAlmacen +
                ", puntoDeReorden=" + puntoDeReorden +
                ", proveedores=" + proveedores +
                ", pedidos=" + pedidos +
                '}';
    }
}
```

Código Clase pedido:

```java

package Almacen.Restaurante;
import java.time.LocalDate;
import java.util.ArrayList;

public class pedido {  3 usages  new *

    private int cantidadPedida;  3 usages
    private int cantidadRecibida;  3 usages
    private LocalDate fechaPedido;  4 usages
    private LocalDate fechaLlegada;  3 usages
    private ArrayList <pedido> pedidos;  no usages

    public pedido(int cantidadPedida, int cantidadRecibida, LocalDate fechaPedido, LocalDate fechaLlegada){  no usages  new *
        this.cantidadPedida = cantidadPedida;
        this.cantidadRecibida = cantidadRecibida;
        this.fechaPedido = fechaPedido;
        this.fechaLlegada = fechaLlegada;

    }
    public LocalDate getfechaPedido(){  1 usage  new *
        return fechaPedido;

    }
    public void actualizarPedido(int cantidadPedida, LocalDate fechaPedido){  no usages  new *
```

```java
public class pedido {  3 usages  new *
    }
    public void actualizarPedido(int cantidadPedida, LocalDate fechaPedido){  no usages  new *
        this.cantidadPedida = cantidadPedida;
        this.fechaPedido = fechaPedido;
    }

    public void registrarLlegada(int cantidadRecibida, LocalDate fechaLlegada){  no usages  new *
        this.cantidadRecibida = cantidadRecibida;
        this.fechaLlegada = fechaLlegada;
    }

    @Override  new *
    public String toString() {
        return "pedido{" +
                "cantidadPedida=" + cantidadPedida +
                ", cantidadRecibida=" + cantidadRecibida +
                ", fechaPedido=" + fechaPedido +
                ", fechaLlegada=" + fechaLlegada +
                '}';
    }
}
```

Código clase proveedor:

mercancia.java pedido.java proveedor.java

```java
package Almacen.Restaurante;
import java.util.ArrayList;
public class proveedor {  2 usages  new *

    private String nombre;  3 usages
    private String telefono;  3 usages
    private String direccion;  3 usages

    public proveedor(String nombre, String telefono, String direccion) {  no usages  new *
        this.nombre = nombre;
        this.telefono = telefono;
        this.direccion = direccion;
    }

    public void mostrarDatosProveedor() {  no usages  new *
        System.out.println("Nombre: " + nombre);
        System.out.println("Telefono: " + telefono);
        System.out.println("Direccion: " + direccion);
    }

    @Override  new *
    public String toString() {
        return "proveedor{" +
                "nombre='" + nombre + '\'' +
                ", telefono='" + telefono + '\'' +
```

```java
mercancia.java   pedido.java   proveedor.java ×

3   public class proveedor {  2 usages  new *
9     public proveedor(String nombre, String telefono, String direccion) {  no usages  new *
13    }
14
15    public void mostrarDatosProveedor() {  no usages  new *
16        System.out.println("Nombre: " + nombre);
17        System.out.println("Telefono: " + telefono);
18        System.out.println("Direccion: " + direccion);
19    }
20
21    @Override  new *
22    public String toString() {
23        return "proveedor{" +
24                "nombre='" + nombre + '\'' +
25                ", telefono='" + telefono + '\'' +
26                ", direccion='" + direccion + '\'' +
27                '}';
28    }
29  }
30
31
```

Prueba códigos:

```
jshell> import AlmacenRestaurante.Mercancia;

jshell> import java.time.LocalDate;

jshell> import java.util.ArrayList;

jshell> Mercancia carne =new Mercancia("Carne de Res",10,LocalDate.now().minusDays(5),15,new ArrayList<>(),new ArrayList<>());
carne ==> AlmacenRestaurante.Mercancia@4b85612c

jshell> import AlmacenRestaurante.proveedor;

jshell> import java.util.ArrayList;

jshell> proveedor p1=new proveedor("Distribuidora Global","555-1234","Av. Central 123");
p1 ==> proveedor{nombre='Distribuidora Global', telefono ... reccion='Av. Central 123'}

jshell> proveedor p2=new proveedor("Carnes Selectas","555-5678","Calle Roble 45");
p2 ==> proveedor{nombre='Carnes Selectas', telefono='555 ... ireccion='Calle Roble 45'}

jshell> import AlmacenRestaurante.Mercancia;
```

Aquí primero importamos la clase mercancía, y creamos una nueva mercancía llamada carne y escribimos todos los datos que declaramos en el código, después importamos la clase proveedor y creamos un nuevo proveedor p1, y escribimos todos los datos que declaramos en el código, después creamos otro proveedor p2 y nuevamente escribimos los datos, y vemos que se guardan los datos de p1 y p2, después importamos la clase mercancía.

```
jshell> import AlmacenRestaurante.Mercancia;

jshell> carne.agregarProveedor(p1);
$10 ==> "Proveedor agregado"

jshell> carne.agregarProveedor(p2);
$11 ==> "Proveedor agregado"

jshell> import AlmacenRestaurante.pedido;

jshell> import java.time.LocalDate;

jshell> import java.util.ArrayList;

jshell> pedido nuevoPedido = new pedido(50, 0, LocalDate.now(), null);
nuevoPedido ==> pedido{cantidadPedida=50, cantidadRecibida=0, fec ... -03-05, fechaLlegada=null}
```

Ya que importamos la clase mercancía agregamos los dos proveedores p1 y p2 a la mercancía carne, después importamos la clase pedido y creamos un nuevo pedido, escribimos los datos necesarios que declaramos en el código y al ejecutar el nuevo pedido vemos que si nos marca la cantidad pedida, la cantidad recibida, vemos que la fecha de llegada esta sin definir por que no se han recibido la cantidad.

Conclusiones:

Durante la practica se nos presentaron problemas igual mente como nos pasó en el desarrollo de las anteriores actividades, y nos dimos cuenta que hay algunos métodos mas complicados que otros ya que requieren mas argumentos que otros y algún otro procedimiento, pero al final pudimos comprender sobre la practica ya que nos ayudo a reforzar nuestros conocimientos sobre la abstracción y el encapsulamiento.

GitHub URL:

https://github.com/soysantiagorb/Practica2

https://github.com/irvingemanuel2003-a11y/practica2.git